JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA
Department of Computer Science & Engineering and Information Technology | Academic Session 2026-27

PROBLEM BASED LEARNING (PBL) SYNOPSIS PRESENTATION

SecureShare

An End-to-End Encrypted Client-Server File Sharing System

Course: Python Lab (Course Code: 26B15CS217)

PROJECT TEAM

Labhansh Vashisht
Enrollment No: 992501230050

Mohd. Nauman Ali
Enrollment No: 992501230060

PROGRAM DETAILS

- **Program:** B.Tech CSE (Cyber Security)
- **Year & Semester:** Second Year / 3rd Semester
- **PBL Domain:** Network Security & Data Privacy
- **Technology Focus:** Python, Sockets, AES & RSA Encryption, SQLite

PROJECT BACKGROUND

1. Problem Statement & Project Objectives

PROBLEM STATEMENT

Challenges in Modern File Sharing:

- **Unsafe Cloud Storage:** Normal cloud services store readable files on servers. If a server is breached or leaked, private files are exposed.
- **No Privacy Verification:** Users cannot easily verify if their files stayed private during transfer or if someone modified them.
- **Security Tools Are Too Complex:** Existing encryption tools (like PGP) are difficult for everyday users, so people avoid encrypting files.

KEY OBJECTIVES

Project Goals & Deliverables:

Safe End-to-End File Sharing: Build an app where files are locked before leaving your computer. The server only stores locked files and cannot read them.

Fast & Strong Encryption: Use AES to lock files quickly, RSA to share unlock keys safely, and SHA-256 to catch any tampering.

Simple Desktop Interface: Create an easy-to-use GUI in Tkinter so any user can send and receive encrypted files with one click.

Core Python Engineering: Use sockets for direct networking, multi-threading for responsiveness, and SQLite to manage user accounts.

SYSTEM DESIGN & HOW IT WORKS

2. Proposed Solution & System Architecture

1. Client Tier (Desktop App)

- Easy-to-use desktop GUI (Tkinter)
- Locks files locally before sending (AES)
- Creates key pairs for secure sharing (RSA)
- Generates file fingerprint to detect changes

2. Server Tier (Network Relay)

- Python socket server manages connections
- Relays messages and file chunks
- Shares public keys between users
- Stores only locked files (cannot read them)

3. Storage Tier (Database & Files)

- Saves locked files safely on disk
- SQLite database for user accounts
- Keeps records of transfers & timestamps
- Protects passwords using secure hashing

HOW SECURE SHARING WORKS (STEP-BY-STEP)

Step 1: Lock the File

Sender locks the file with a one-time secret key (AES). The file turns into scrambled data.

Step 2: Lock the Key

The secret key is locked with the receiver's public key (RSA). Only the receiver can open it.

Step 3: Create Fingerprint

A unique digital fingerprint (SHA-256) of the original file is made to catch any tampering.

Step 4: Unlock & Check

Receiver unlocks the secret key with their private key, restores the file, and confirms the fingerprint.

PROJECT ROADMAP & PHASES

3. Methodology & 8-Phase Implementation Plan

Phase 1: Socket Networking
Set up direct communication between clients and the server using Python sockets.

Phase 2: User Accounts & Login
Build sign-up and login features with securely hashed passwords saved in SQLite.

Phase 3: File Encryption (AES)
Lock files on the sender's computer before sending, splitting large files into safe chunks.

Phase 4: Key Sharing (RSA)
Create public and private key pairs so users can safely exchange file unlock keys.

Phase 5: Tamper Detection (SHA-256)
Check file fingerprints before and after transfer to guarantee files were not altered.

Phase 6: Desktop App UI
Design a simple screen where users can pick files, choose recipients, and see transfer status.

Phase 7: Transfer History Database
Store user lists, sent/received file history, and transfer dates in SQLite.

Phase 8: Security Testing & Demo
Test the system against file tampering and network eavesdropping to prove files stay safe.

TOOLS & REAL-WORLD IMPACT

4. Technology Stack & SDG Mapping

PROPOSED TECH STACK

- **Python 3.8+ [Language]:** Main programming language; clean, reliable, and easy to build with.
- **socket (Standard Library) [Networking]:** Connects client and server to transfer files directly.
- **cryptography (pyca) [Security]:** Standard library used for AES (file locking) and RSA (key sharing).
- **hashlib (Standard Library) [Tamper Check]:** Generates SHA-256 fingerprints to verify file integrity.
- **sqlite3 [Database]:** Lightweight database to store user accounts and transfer logs.
- **tkinter [Desktop UI]:** Built-in Python library used to build a simple graphical interface.

PRIMARY SDG: SDG 9

Industry, Innovation and Infrastructure

- Builds dependable digital tools for transferring files safely without risk of data theft.
- Makes strong encryption simple and accessible for everyday communication.

SECONDARY SDG: SDG 16

Peace, Justice and Strong Institutions

- Protects personal privacy by preventing eavesdropping and unauthorized data access.
- Ensures file sharing is transparent, accountable, and tamper-proof.

DELIVERABLES & ROADMAP

5. Project Timeline, Expected Outcomes & Conclusion

PROJECT TIMELINE

- **Week 1 (Sept 7 - 8): System Design**
  Plan client-server communication, database tables, and project structure.
- **Week 2 (Sept 9 - 11): Network & File Encryption**
  Build client-server socket connection, login system, and AES file locking.
- **Week 3 (Sept 12+): Key Sharing & GUI**
  Add RSA key sharing, tamper check (SHA-256), and desktop UI.
- **Week 4 (Final): Testing & Final Demo**
  Save transfer logs, perform security demo (tamper check), and prepare report.

EXPECTED OUTCOMES

Working Secure File App

Complete app where only sender and receiver can view files; server never reads data.

Clean & Organized Python Code

Well-structured code using sockets, standard cryptography, and SQLite.

Live Security Demo

Live demonstration showing how tampering is detected and intercepted files cannot be read.

Thank You! | Open for Q&A

Labhansh Vashisht (992501230050) & Mohd. Nauman Ali (992501230060)